Building a RAG-Powered Test Case Generation & Evaluation Framework

As LLM adoption grows in QA and automation, generating accurate, contextual, and production-ready test cases becomes challenging. This framework applies Retrieval-Augmented Generation (RAG) specifically for intelligent test case generation and combines it with automated DeepEval-based quality measurement.

Part 1: RAG for Intelligent Test Case Generation

- Vector store built using historical test cases, requirements, user stories, acceptance criteria, domain validation rules, and defect patterns.
- New requirements converted into embeddings for semantic similarity search.
- Top-K relevant historical scenarios retrieved and ranked.
- Structured prompt constructed using system instructions + retrieved patterns + domain constraints + new requirement.
- Generates positive, negative, boundary, and edge-case scenarios.
- Produces structured, traceable, and automation-ready test steps aligned with past testing strategy.

Part 2: DeepEval-Based Quality Measurement

- Evaluates test case relevancy and faithfulness to the requirement.
- Measures context precision and hallucination risk.
- Detects potential bias in generated outputs.
- Creates LLMTestCase abstraction for regression tracking and benchmarking.
- Enables CI/CD quality gating with threshold-based validation.

End-to-End Flow

Requirement → Embedding → Vector Search → Context Builder → LLM Generates Test Cases → DeepEval Evaluates → Score Report Logged → CI/CD Gate Applied

Business Impact

- Faster and scalable AI-assisted test case authoring.
- Improved coverage through historical pattern retrieval.
- Reduced hallucinations and higher reliability.
- Consistent automation standards across teams.
- Enterprise-grade AI governance and measurable quality control.